

PRD: Drag-and-Drop Canvas on tldraw + Fal + Nano Banana + GLSL + 3D Viewer

1) Overview

A Chrome-only single-user creative canvas using **tldraw** as the editing engine. Users can:

- Drag and drop images
- Move, scale, rotate with handles
- Draw with a brush
- Add shapes and text
- Generate images via **Fal** → **Nano Banana**
- Manage a horizontal gallery with lightbox and delete
- Apply GLSL effects and add a 3D viewer window inside the canvas
- Export the entire composition as a single raster image

Overflow is clipped at the canvas frame. Everything visible in the frame exports.

2) Why tldraw for this

- Built-in infinite canvas, zoom, pan, selection, handles, bounds, hit-testing
- Strong shape model with custom shapes via **TLSHapeUtil**
- Persistent store and command history that maps well to undo/redo
- DOM centric rendering means fast UI work and quick iteration

Key tradeoff: DOM + SVG rendering requires a deliberate **export compositor** to rasterize the final image and to integrate GLSL or 3D content. We solve this with a predictable export pipeline described below.

3) Core feature set

3.1 Canvas and transforms

- Base is a **tldraw Editor** with infinite canvas and a visible frame
- Zoom and pan use tldraw defaults
- Selection uses stock marquee and handles
- Right click context menu: Bring forward, send backward, bring to front, send to back, delete, duplicate

3.2 Objects

Use tldraw shapes where possible:

- **Image shape** for dropped or AI generated images
- **Geo shapes** for rectangle, circle, triangle
- **Draw shape** for brush strokes
- **Text shape** for canvas-rendered text with size and color

Add two custom shapes:

- `shaderShape` for masked GLSL regions
- `viewer3DShape` for a 3D window

Both are implemented as **custom TLShapeUtil** with their own render and bounds.

3.3 Prompt box and Fal

- Text input below the canvas
- On submit: `POST https://fal.run/fal-ai/nano-banana/api` with prompt, size, num_images
- On response: insert image as an **Image shape** at canvas center and into the gallery

3.4 Gallery

- Horizontal scroll strip below the prompt
- Clicking thumbnail opens lightbox modal with Close
- Three dots menu per item with Delete
- Drag from gallery to canvas inserts as Image shape

3.5 Brush, shapes, text

- Use tldraw built-in tools
- Text properties: size and color
- Shapes: fill and stroke settings
- Brush: stroke size, color, opacity

3.6 Ripple on drop

- Play a visual ripple when an asset lands
- Implement as a **fullscreen shader pass overlay** or a **temporary shaderShape** spawned at drop point with timed removal

3.7 Export

- Export button builds a flattened PNG
- Export compositor draws shapes in z order into an **offscreen HTMLCanvasElement**
- For shader shapes and the 3D viewer, render offscreen with WebGL2 or Three.js at the correct pixel size, then draw results into the export canvas
- Everything visible inside the frame is included; outside is clipped

4) Architecture

4.1 Stack

- **tldraw** as editor and scene model
- **React + TypeScript**
- **Zustand** for app UI state that sits beside tldraw store
- **Axios** for Fal
- **Tailwind** for UI chrome
- **WebGL2** (regl or twgl) for shader passes
- **Three.js** for 3D viewer

4.2 Shape model sketch

```
// Image
type ImageShape = TLBaseShape<'image', {
  src: string
  w: number
  h: number
}>

// Shader region
type ShaderShape = TLBaseShape<'shader', {
  mode: 'fullscreen' | 'masked' | 'object'
  passes: Array<{ name: string; fragment: string; vertex?: string;
uniforms: Record<string, any> }>
}>

// 3D viewer
type Viewer3DShape = TLBaseShape<'viewer3d', {
  src: string          // glTF/GLB URL
  bg: 'transparent' | 'color'
  bgColor?: string
  camera: { fov: number; near: number; far: number; orbit: [number,
number, number]; target: [number, number, number] }
```

```
controls: { orbit: boolean; pan: boolean; zoom: boolean; damping:
boolean }
lighting: { env?: string; intensity?: number }
}>
```

4.3 Rendering strategy

- **Editor rendering:** let tldraw render all default shapes in DOM/SVG
- **Shader and 3D overlays:** absolute-positioned canvases in an overlay layer that sync to shape bounds
 - For `shaderShape` with masked mode, the overlay canvas size equals the shape's bounds and uses `overflow: hidden`
 - For `viewer3dShape`, mount a Three.js renderer in an overlay `<div>` with `overflow: hidden` and sync to bounds

This keeps realtime effects smooth without fighting tldraw's DOM. Export uses offscreen rendering for a single flattened image.

4.4 Export compositor

1. Create an offscreen 2D canvas at export resolution
2. Visit shapes in z order
3. For tldraw native shapes:
 - Rasterize using the built-in export utility or re-draw vector primitives into the export canvas
4. For image shapes: draw `ImageBitmap` directly
5. For shader shapes: run WebGL pass offscreen with the correct input and draw the result rect into the export canvas
6. For 3D viewer: render Three.js scene to an `OffscreenCanvas` then draw result into the export canvas
7. Return PNG blob and data URL

5) GLSL integration

5.1 Overlay approach (realtime)

- Add `FXManager` that maintains one WebGL2 context for fullscreen and one per masked shader shape if needed
- Uniforms to support immediately:
 - `uResolution` in pixels
 - `uTime` in seconds

- Pointer uniforms like `uDrop` in screen pixels
- Stage snapshot is optional. Prefer procedural effects for realtime. If you must sample the stage, snapshot only on changes

5.2 Offscreen approach (static or export)

- Render shader pass to `OffscreenCanvas` with WebGL2
- `transferToImageBitmap()` and draw into export canvas or into a temporary `<img>` for preview

5.3 Minimal ripple shader fragment

```
precision highp float;
uniform vec2 uResolution;
uniform float uTime;
uniform vec2 uDrop; // pixels
void main() {
    vec2 p = gl_FragCoord.xy - uDrop;
    float r = length(p);
    float wave = 0.5 + 0.5 * cos(10.0 * (r - 180.0 * uTime));
    float env = smoothstep(220.0, 0.0, r);
    float v = wave * env * 0.35;
    gl_FragColor = vec4(vec3(v), v);
}
```

6) 3D viewer integration

6.1 Overlay approach

- For each `viewer3dShape`, mount a Three.js renderer into an absolutely positioned container that matches the shape's bounds. Use `overflow: hidden`
- OrbitControls inside the bounds
- Pause render loop when viewer not visible or idle for >300 ms

6.2 Export

- Render at the final pixel size using an offscreen WebGL context in Three.js
- Draw to export canvas at the viewer rect

7) UX details

- **Handles:** rely on tldraw selection handles
- **Clipping:** the app shows a canvas frame rectangle. Everything outside is visually hidden. Shapes can exist beyond but export clips to the frame
- **Ripple:** on image drop or AI insert, create a ripple at the drop point. Implement either as temporary shader overlay or as a short-lived shaderShape
- **Shortcuts:**
 - V Move
 - B Brush
 - R Rectangle
 - C Circle
 - T Text
 - A Triangle
 - Delete removes selection
 - Cmd or Ctrl + Z undo
 - Shift + Cmd or Ctrl + Z redo

8) Performance targets

- 60 fps interaction with up to 10 large images and light shader activity
- Throttle shader overlays when idle
- Avoid frequent stage snapshots
- Use `createImageBitmap` for large images
- Handle devicePixelRatio properly for crisp output

9) Error handling

- Unsupported drops show a toast
- Fal errors show retry with preserved prompt
- Autosave tldraw store to localStorage every 5 seconds with restore banner

10) Milestones

Milestone	Scope
M1	tldraw editor with images, shapes, text, brush, layer ordering, undo/redox
M2	Prompt box + Fal integration + insert as Image shape
M3	Gallery + lightbox + drag to canvas
M4	Export compositor that flattens native shapes + images

- M5 Shader overlays with ripple on drop. Offscreen shader for export
- M6 3D viewer overlay. Offscreen 3D render for export
- M7 Performance passes and polish

11) Implementation stubs

11.1 Register custom shapes

```
// shaderShape.util.ts
export class ShaderShapeUtil extends TLShapeUtil<ShaderShape> {
  static type = 'shader' as const
  getDefaultProps() { return { mode: 'masked', passes: [] } }
  component(shape) {
    return <div data-shape="shader" /> // overlay will draw actual
pixels
  }
  indicator(shape) { return null }
}

// viewer3dShape.util.ts
export class Viewer3DShapeUtil extends TLShapeUtil<Viewer3DShape> {
  static type = 'viewer3d' as const
  getDefaultProps() { return { src: '', bg: 'transparent', camera: {
fov: 50, near: .1, far: 1000, orbit: [0,0,3], target: [0,0,0] },
controls: { orbit: true, pan: true, zoom: true, damping: true },
lighting: { intensity: 1 } } } }
  component(shape) {
    return <div data-shape="viewer3d" /> // overlay mounts Three.js
here by rect
  }
  indicator(shape) { return null }
}
```

11.2 Overlay sync

```
function useShapeScreenRect(editor: Editor, id: TLShapeId) {
  const pageBounds = editor.getShapePageBounds(id)
```

```

const zoom = editor.getZoomLevel()
const camera = editor.getCamera()
const sx = pageBounds.x * zoom - camera.x
const sy = pageBounds.y * zoom - camera.y
const sw = pageBounds.w * zoom
const sh = pageBounds.h * zoom
return { left: sx, top: sy, width: sw, height: sh }
}

```

11.3 Export compositor outline

```

async function exportToPNG(editor: Editor, frameId: TLShapeId, scale =
2) {
  const frame = editor.getShapeById(frameId)!
  const b = editor.getShapePageBounds(frameId)
  const off = new OffscreenCanvas(b.w * scale, b.h * scale)
  const ctx = off.getContext('2d')!

  const shapes = editor.getCurrentPageShapesSorted()
  for (const s of shapes) {
    if (!intersects(b, editor.getShapePageBounds(s.id))) continue

    if (s.type === 'image') {
      const img = await loadBitmap(s.props.src)
      const rect = localRectInFrame(editor, s.id, frameId, scale)
      ctx.drawImage(img, rect.x, rect.y, rect.w, rect.h)
    } else if (s.type === 'shader') {
      const bmp = await renderShaderOffscreen(s, rectSize(s, scale))
      ctx.drawImage(bmp, rectX(s, frameId, scale), rectY(s, frameId,
scale))
    } else if (s.type === 'viewer3d') {
      const bmp = await renderViewerOffscreen(s, rectSize(s, scale))
      ctx.drawImage(bmp, rectX(s, frameId, scale), rectY(s, frameId,
scale))
    } else {
      // Rasterize native vector primitives or use tldraw's SVG export
      then drawImage
    }
  }
}

```



```
        const bmp = await rasterizeNativeShape(editor, s, frameId,
scale)
        ctx.drawImage(bmp, 0, 0)
    }
}
return await off.convertToBlob({ type: 'image/png' })
}
```

If you want, I can follow this with a **tldraw starter scaffold** for Cursor:

- Editor setup with custom `shaderShape` and `viewer3dShape` registered
- `FXManager` overlay with a working ripple pass
- Fal client and gallery strip
- Export compositor that handles native shapes, images, one shader region, and one 3D viewer window

You said:

no fal integration mentioned. add **EVERYTHING** that is needed

ChatGPT said:

Thought for 2m 27s

Stigma-like Canvas Prd — Tldraw + Fal + Nano Banana

Stigma-like Canvas PRD (tldraw + FAL + Nano Banana)

Status: Draft for build in Cursor. Scope is V1 unless marked V2+.

1. Product summary

A browser canvas app for fast visual ideation. Users drag images onto a zoomable, pannable tldraw canvas, arrange and transform them, sketch with a lightweight brush, drop simple shapes, add text, then flatten the whole canvas to a raster and generate new images through Nano Banana on FAL. Each generation appears in a horizontal gallery below the canvas with lightbox and delete. Right-click on any placed image exposes layer order controls.

2. Goals and non-goals

- Goals
 - Frictionless drag-and-drop of images from desktop and web
 - Snappy transforms: move, scale, rotate with precision handles
 - Simple creation tools: Brush, Shapes, Text
 - Reliable flatten-and-send to FAL's Nano Banana text-to-image and edit endpoints
 - Clean gallery with lightbox and basic management
 - Delightful drop ripple animation on the canvas
 - Robust error handling, retry, and queue feedback from FAL
- Non-goals
 - Deep layer styles, complex vector ops, masks for model guidance (future)
 - Multi-user collaboration (future)
 - Export to PSD or SVG (future); V1 exports PNG/JPEG/WebP only

3. Key workflows

3.1 Drag and drop

1. User drags one or more files onto canvas.
2. Files import as **ImageShape** nodes at cursor position.
3. On drop, show a GPU ripple animation.
4. Images are clipped to the canvas bounds. No outside-opacity effect.

3.2 Transform

- Select any shape (image, rect, ellipse, triangle, text, brush stroke group)
- Move by dragging anywhere on the selection
- Scale uniformly from corners, free scale from edges
- Rotate from an outside handle
- Right-click context menu on image shapes: Bring to Front, Send to Back

3.3 Create

- Brush tool: size and color controls, freehand strokes grouped as a single shape; performance budget for thousands of points
- Shapes tool: rectangle, ellipse, triangle; fill color, stroke color, stroke width
- Text tool: click to create text; size and color controls; edit in place

3.4 Prompt and generate (Nano Banana on FAL)

- Prompt input sits directly under the canvas
- Actions
 - Generate from text: sends prompt only to Nano Banana t2i

- Edit from canvas: flattens entire canvas to a single image and sends prompt + that image to Nano Banana edit
- Queue UI
 - Inline status chip per job: In queue, Running, Completed, Failed
 - Cancel while queued

3.5 Gallery

- Horizontal gallery row under prompt
- Each generation shows 1 or more thumbnails; clicking opens a lightbox modal
- Item menu: Delete; Regenerate can be V2

4. UX specifics

- Canvas interaction
 - Wheel to zoom, Space+drag to pan
 - Snap to pixel when zoom $\geq 600\%$ to keep crisp edges
 - Selection box shows white circular handles with black outline, black bounding outline
 - Holding Shift constrains scale uniform; Option clones on drag; R rotates in 15° increments
- Brush HUD
 - Number keys 1–5 set brush sizes
 - Hold B to temporarily switch to Brush
- Shapes HUD
 - Inline palette with recent colors; use HSL sliders
- Text HUD
 - Font size slider, color picker; basic alignment left/center/right
- Ripple animation on drop
 - Subtle concentric waves, ~300 ms total, easing in and out; disabled while panning

5. Information architecture

- Top bar: App title, Export, Settings
- Left rail: Move, Brush, Shapes, Text, Select, Hand tools
- Right rail: Context inspector for the active tool or selection
- Bottom: Prompt input, Generate buttons, Job status, Gallery

6. Data model (TypeScript)

```
// Canvas
export type TLDoc = {
```

```

id: string;
shapes: TLShapeRecord[]; // tldraw records
camera: { x: number; y: number; z: number };
updatedAt: number;
};

export type GalleryItem = {
id: string;
requestId: string; // FAL queue id
createdAt: number;
prompt: string;
source: 'text' | 'canvas-edit';
images: { url: string; width?: number; height?: number; contentType?: string }[];
description?: string; // returned by Nano Banana
status: 'queued' | 'running' | 'ok' | 'error';
error?: { type: string; msg: string; retryable?: boolean };
};

export type FalJob = {
id: string; // mirrors requestId
endpoint: 'fal-ai/nano-banana' | 'fal-ai/nano-banana/edit';
input: Record<string, unknown>;
statusUrl?: string; // queue status url
};

```

7. Tech stack

- React 18 + Next.js App Router
- tldraw v2 for canvas primitives and custom shapes
- WebGL2 for ripple shader, with graceful fallback to CSS animation
- Node runtime API routes for secure FAL calls, including queue and webhook handling

8. Architecture

8.1 Canvas and shapes

- Use tldraw core shapes for rectangle, ellipse, text, and ImageShape
- Custom Triangle shape using a polygon path
- Brush strokes: a single shape that stores a polyline; render as signed distance field or stroke tessellation
- All shape renders respect canvas clipping. No bleed outside bounds.

8.2 Flattening

- Export via `toCanvas` for all TL shapes to an `OffscreenCanvas` at target size
- Option: set export size by aspect ratio selector in prompt row
- Resulting bitmap is encoded as PNG or WebP for edit flow

8.3 Ripple shader

- `OnDrop`: obtain drop position in view coordinates
- Run a fragment shader for a short burst over a full screen quad in an overlay WebGL canvas
- Shader parameters: center, time, amplitude, wavelength; total duration ~300 ms
- Fallback: CSS radial gradient keyframes when WebGL not available

8.4 FAL proxy and queue

- All calls to FAL originate on server routes
- Client posts `/api/fal/t2i` and `/api/fal/edit`
- Server uses `@fal-ai/client` queue methods; stores `request_id` and returns to client for live status polling or webhook completion
- Optionally configure webhooks to push job completion to `/api/fal/webhook`

9. FAL and Nano Banana integration

9.1 Security and setup

- Environment variables
 - `FAL_KEY` set in server environment only
- Client config for realtime or websocket features will use a proxy URL

9.2 Client to server requests

```
// POST /api/fal/t2i
// body: { prompt: string; aspectRatio?: string; numImages?: number; outputFormat?:
'jpeg'|'png'|'webp' }
// returns: { requestId: string }

// POST /api/fal/edit
// body: { prompt: string; imageUrl: string; aspectRatio?: string; numImages?: number;
outputFormat?: 'jpeg'|'png'|'webp' }
// returns: { requestId: string }
```

9.3 Server implementation sketch

```
// app/api/fal/t2i/route.ts
import { NextRequest, NextResponse } from 'next/server';
```

```

import { fal } from '@fal-ai/client';

export async function POST(req: NextRequest) {
  const { prompt, aspectRatio = '1:1', numImages = 1, outputFormat = 'jpeg' } = await req.json();
  const { request_id } = await fal.queue.submit('fal-ai/nano-banana', {
    input: { prompt, aspect_ratio: aspectRatio, num_images: numImages, output_format:
    outputFormat },
  });
  return NextResponse.json({ requestId: request_id });
}
// app/api/fal/edit/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { fal } from '@fal-ai/client';

export async function POST(req: NextRequest) {
  const { prompt, imageUrl, aspectRatio, numImages = 1, outputFormat = 'jpeg' } = await
  req.json();
  const { request_id } = await fal.queue.submit('fal-ai/nano-banana/edit', {
    input: { prompt, image_urls: [imageUrl], ...(aspectRatio ? { aspect_ratio: aspectRatio } : {}),
    num_images: numImages, output_format: outputFormat },
  });
  return NextResponse.json({ requestId: request_id });
}
// app/api/fal/status/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { fal } from '@fal-ai/client';

export async function POST(req: NextRequest) {
  const { endpoint, requestId } = await req.json();
  const status = await fal.queue.status(endpoint, { requestId, logs: true });
  return NextResponse.json(status);
}
// app/api/fal/result/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { fal } from '@fal-ai/client';

export async function POST(req: NextRequest) {
  const { endpoint, requestId } = await req.json();
  const result = await fal.queue.result(endpoint, { requestId });
  return NextResponse.json(result);
}

```

9.4 Webhook support (optional but recommended)

- Configure queue submit with `webhookUrl`:
`https://your.app/api/fal/webhook`
- Implement signature verification when available, and update the gallery item on payload receipt

```
// app/api/fal/webhook/route.ts
import { NextRequest, NextResponse } from 'next/server';

export async function POST(req: NextRequest) {
  const raw = await req.text();
  // Optionally verify X-Fal-Webhook-* headers per docs
  const data = JSON.parse(raw);
  // upsert into store by data.request_id; set images, status
  return NextResponse.json({ ok: true });
}
```

9.5 File upload and data URIs

- When sending the flattened canvas for edit, first upload the `Blob` via `fal.storage.upload(file)` to get a public URL, then pass it as `image_urls: [url]` to the edit endpoint. This avoids large base64 payloads in the queue body.

9.6 Endpoint schemas to respect

- Text to Image `fal-ai/nano-banana`
 - Required: `prompt`
 - Optional: `num_images`, `output_format` (jpeg, png, webp), `aspect_ratio` (21:9, 1:1, 4:3, 3:2, 2:3, 5:4, 4:5, 3:4, 16:9, 9:16), `sync_mode`
- Image to Image `fal-ai/nano-banana/edit`
 - Required: `prompt`, `image_urls: string[]`
 - Optional: `num_images`, `output_format`, `aspect_ratio`, `sync_mode`, `limit_generations`

9.7 Transport choices

- Prefer queue submit + webhook or queue polling for reliability
- Synchronous `fal.run` is reserved for very fast calls only
- Realtime over WebSockets is not needed for V1; keep proxy route in place to enable later

9.8 Error handling strategy

- Parse fal error responses; map **type** to user-friendly messages
- If **X-Fal-Retryable** is true, surface a Retry button on failures
- Handle common image issues gracefully: too small, too large, file download errors

10. Controls and settings

- Export: PNG, JPEG, WebP; select resolution and aspect ratio
- Canvas settings: grid on or off, snapping, background color
- Model settings: aspect ratio, count, format; advanced hidden behind a Details expander

11. Accessibility

- Tool icons have labels and ARIA roles
- Keyboard navigation across tools and shapes; Escape clears selection; Delete removes
- High contrast theme toggle

12. Telemetry

- Events: drop_image, transform_done, brush_stroke_end, t2i_submit, edit_submit, job_completed, job_failed, gallery_open, lightbox_open, export
- Performance marks: flatten_time_ms, upload_time_ms, queue_wait_ms, inference_ms

13. Performance budgets

- Flatten to 2048 px longest side within 150 ms on M1 class machines
- Brush maintains 60 fps for 2k points strokes
- Gallery first paint under 50 ms for 50 thumbs

14. Future work

- GLSL stackable filters per shape
- 3D viewer window inside canvas using three.js; serialize camera and viewport as a dynamic shape; render into the flatten pass
- Regenerate from gallery item
- Multi-select and group, auto-layout grid

15. Acceptance criteria

- Drag-and-drop imports single and multiple images, with ripple animation, clipped at bounds

- Right-click context menu changes z-order
- Brush draws with adjustable size and color; Shapes and Text operate with stated controls
- Prompt to t2i returns images into gallery; Edit from canvas sends flattened image and returns result
- Gallery lightbox works with keyboard nav and close controls
- Error states and retries behave as specified

16. QA checklist

- Drop large images, tiny images, and broken URLs, verify errors
 - Cancel queued job, ensure it does not produce a gallery item
 - Offline mode while a job is in progress, then back online, result reconciliation via polling
 - Verify export sizes and aspect ratios
-

Appendix A: Minimal UI contracts

// Canvas flatten

```
async function flattenCanvasToBlob(doc: TLDoc, px: number, format:
'image/png'|'image/webp'|'image/jpeg'): Promise<Blob> { /* ... */ }
```

// Ripple trigger

```
function playDropRipple(viewXY: {x:number;y:number}): void { /* ... */ }
```

// FAL API client

async function

```
submitT2I(input:{prompt:string;aspectRatio?:string;numImages?:number;outputFormat?:'jpeg'|'p
ng'|'webp'}): Promise<{requestId:string}> { /* fetch('/api/fal/t2i', ...) */ }
```

async function

```
submitEdit(input:{prompt:string;imageUrl:string;aspectRatio?:string;numImages?:number;output
Format?:'jpeg'|'png'|'webp'}): Promise<{requestId:string}> { /* fetch('/api/fal/edit', ...) */ }
```

// Poll or subscribe

```
async function getStatus(endpoint:string, requestId:string): Promise<any> { /* /api/fal/status */ }
```

```
async function getResult(endpoint:string, requestId:string): Promise<any> { /* /api/fal/result */ }
```

Appendix B: GLSL ripple fragment (concept)

precision mediump float;

uniform vec2 u_res; // canvas size

uniform vec2 u_center; // drop position in px

uniform float u_time; // 0..1 animation

```
void main(){
vec2 uv = gl_FragCoord.xy / u_res;
vec2 c = u_center / u_res;
float d = distance(uv, c);
float wave = sin((d - u_time) * 40.0) * exp(-6.0 * d);
float alpha = smoothstep(0.0, 0.02, wave);
gl_FragColor = vec4(vec3(1.0), alpha * 0.25);
}
```

Appendix C: Notes on masks

- Nano Banana edit does not expose an explicit mask parameter in its schema. The brush is a creative sketch tool that becomes part of the flattened image. If masked inpainting is needed later, use a model that accepts mask inputs or composite locally before sending.